Vellore Institute of Technology

IT Department

Aethor OS

An Observable Operating Systems Simulator with SpillGuard GPU Memory Diagnostics

OPERATING SYSTEMS LABORATORY
PROJECT REPORT

Submitted by
Varad Patil
Registration Number 24BIT0175

Individual Project

Course Code
BITE303P

Faculty
Dr. Harshita Patel

22 October 2026

Abstract

Aethor OS, named AetherOS in the source tree and interface, is a browser-based operating systems laboratory that connects interactive simulation with a native Windows GPU-memory diagnostic workflow. Its TypeScript kernel models process dispatch on two logical cores, scheduling policies, frame allocation, page replacement, resource-allocation safety, wait-for cycles, a virtual file system, and event probes. A React desktop exposes the model through process tables, Gantt histories, memory views, a shell, and diagnostic applications. The browser kernel is an educational model; it does not boot hardware or execute native user programs.

The SpillGuard extension collects process and adapter memory observations through Windows PDH, DXGI, optional NVML, and system-process information. It provides memory-pressure alerts, a model-footprint estimator, admission advice, trace replay, and controlled baseline–pressure–recovery experiments. Saved measurements on one RTX 3050 6 GB laptop show a reproducible performance cliff for a fixed Qwen2.5 3B inference workload: planned overflow of 256 MiB produced 2.35–2.37 times baseline decode time in the initial two trials. A subsequent remedy experiment reproduced the witness and reduced slowdown to approximately 1.01 times baseline by pausing the owned co-tenant while retaining its allocation.

The result establishes a local performance failure under a controlled intervention. It does not establish a universal residency mechanism. Similar memory counters accompanied both severe and benign outcomes, demonstrating the importance of work-preserving intervention and recovery checks. Current software verification comprises 66 passing frontend tests, 149 passing agent tests, and a successful production build. Remaining limitations include simplified simulator semantics, small hardware samples, a single validated real application target, and unvalidated empirical advantages of scenario reduction.

Keywords Operating systems education; CPU scheduling; paging; deadlock; virtual file systems; GPU memory; Windows telemetry; performance diagnosis; reproducibility.

Reading the report

Chapters 1–3 establish the problem, objectives, background and methodology. Chapters 4–7 describe the architecture and implementation of the simulator and native agent. Chapters 8–10 present the experimental method, software tests, results and limitations. Chapters 11–12 provide the user manual, conclusions and future enhancements. References identify the theoretical sources and retained project evidence.

Novelty and Original Contribution

Contribution statement. SpillGuard contributes a Windows-focused diagnostic workflow that connects process-level GPU-memory observations to controlled local-LLM slowdown experiments, validated remedies and replayable evidence inside an interactive OS laboratory. Its originality is supported as an implemented systems integration and a measured diagnostic case study. The evidence does not establish a first-ever spill detector, a new scheduling algorithm or universal research priority.

N1 From a pressure alert to a validated incident

A high memory counter describes pressure, but does not establish how much useful work has slowed. SpillGuard Repro freezes the model, prompt, context and output work; runs baseline, pressure and recovery; checks output hashes and observed manipulation; and records PASS, FAIL or INCONCLUSIVE. This turns an alert into a testable incident record. Causal profiling and counterfactual debugging are established [20, 21]; the project contribution is their limited experimental adaptation to owned co-tenant activity in the Windows local-inference workflow. [9, 12]

N2 Keeping allocation while changing activity

The retained pilot shows comparable displayed memory counters with a 2.37-times slowdown for an active co-tenant and approximately 1.03 times baseline for an idle co-tenant. A subsequent remedy retained the 3,709 MiB allocation while pausing activity, producing ratios of approximately 1.01 in two matched trials. This demonstrates why occupancy and harmful interference must be distinguished in this workload. It is an empirical contribution of this implementation and dataset; first discovery of the general phenomenon is not claimed. [9, 11, 26]

N3 Connecting resource safety to native observation

The platform places simulated paging, resource matrices and process dispatch alongside live or recorded host memory. Its admission view compares textbook Banker completion assumptions with reservations for long-lived applications. Python and browser implementations expose this reasoning during replay. The reservation is a context-specific adaptation of established resource-allocation principles, not a new safety theorem. The educational contribution is the inspectable connection from an OS concept to a native memory decision; learning gains have not yet been measured. [1, 6, 7, 25]

Defensible novelty claim. The project combines an OS teaching environment with an evidence-preserving Windows GPU-memory diagnosis workflow and demonstrates a local case where changing co-tenant activity restores inference performance without freeing its allocation. This is a concrete project contribution. Superiority over existing profilers and global uniqueness remain unestablished.

Novelty Comparison with Existing Work

This comparison identifies established capabilities before stating the project difference. Papers are compared by their described contributions; official tool documentation establishes available features. No competitor was executed under a matched benchmark. A different platform or interface alone is insufficient to establish research novelty.

Comparison N1 Closest published methods and documented tools

Existing work	**Already established**	**SpillGuard distinction and limit**
Nsight [14]; hypomnesis [15]; VGPU-Mon [27]	Windows paging timelines, saturation/shared-growth detection and per-process GPU telemetry.	Adds frozen-work records, owned remedy trials and OS-lab replay. Monitoring and the detector principle are not new.
PILOT [16]; Nixie [17]	Oversubscription mitigation using memory-aware scheduling; transparent consumer-GPU multiplexing.	Diagnoses an unchanged target rather than replacing GPU scheduling. Pausing and serializing competing work are established remedies.
CONCUR [18]; PagedAttention [23]	KV-pressure-aware admission and efficient KV-cache management.	Explains host VRAM/commit reservations and long-lived claims. It does not invent KV paging or establish a new admission policy.
DrGPUM [19]	Attributes memory inefficiencies to objects and APIs and guides optimization.	Uses process-level intervention and recovery rather than object profiling. Greater diagnostic precision is not demonstrated.
Coz [20]; CADET [21]	Performance interventions, causal profiling and counterfactual diagnosis and repairs.	Adapts controlled activity changes to Windows inference. It does not invent causal diagnosis or verify a residency mechanism.
GGUF Parser [22]	Estimates memory with context, layer offload and runtime-related configuration.	Adds local calibration and observed-holder admission context. The weights-plus-KV-plus-buffer decomposition is established.
Delta debugging [5]; ProbDD [24]	Input reduction while preserving a property, with informed candidate selection.	Implements bounded transforms and proxy checks. Real-witness reduction benefit and global minimality remain unproven.
Buck and Perugini [25]	Interactive OS scheduling simulation through a React web application.	Connects the simulator to native GPU diagnostics and evidence replay. React, Gantt charts and visualization are established.

Assessment. The supported originality lies in the implemented diagnostic task and its evaluated local example. The reviewed sources do not establish that the exact integration is globally unique. Living tool documentation is compared as accessed on 24 October 2026; feature-level priority dates were not independently audited.

Evidence Supporting the Novelty Claim

Comparison N2 Contribution evidence and scope

Contribution	Project evidence	What it establishes
Incident workflow	Frozen recipes; output hashes; recovery; tri-state outcomes; SHA-256 bundles. [12]	Executable, evidence-preserving experimental plumbing. Hashes do not establish authenticity or external replication.
Capacity-boundary case	Two trials per active dose; +256 MiB ratios 2.37/2.35; +512 MiB ratios 2.38/2.28. [9, 11]	A local performance cliff under the frozen workload, not population significance.
Allocation-preserving remedy	Retain 3,709 MiB; pause touches; ratios approximately 1.01/1.01; unchanged output work. [9]	An effective remedy for the owned co-tenant. Paging causality and general third-party remedies remain unverified.
Explainable OS integration	Live/replay views; textbook Banker and long-lived reservation comparison. [6, 7]	Implemented integration and explanatory capability, not a new theorem or measured learning improvement.

Why the empirical distinction matters

On nine valid dose/idle trials, the original counter detector alerted on all four severe failures and on three of five non-failures: precision was 4/7 for this severe-slowdown label. The contribution is not perfect detection. Matched experiments supply information that the counter alert lacks, while recovery and fixed-work checks strengthen the incident record. The active/idle contrast motivates residency-event verification; it does not itself prove repeated page migration. [9, 11, 26]

Research scope and stronger validation

The targeted review screened 42 search-result candidates across six searches covering Windows diagnostics, GPU scheduling, causal diagnosis, reduction, model estimation and OS simulators. Selected primary manuscripts, author abstracts and official documentation support the comparisons [14–27]. This is not an exhaustive proof that no identical system exists. External literature establishes what is already known; the project experiments establish its own results.

A stronger publication-level claim requires matched diagnostic-baseline comparisons, more models and hardware, independent residency measurements, and reducer evaluation on the real positive witness under equal budgets. Current evidence supports a distinctive engineering integration and bounded empirical finding. It does not establish a globally first detector, a novel general-purpose algorithm, patentability or unmeasured superiority.

Contents

Abstract 2

Novelty and Original Contribution 3

Novelty Comparison with Existing Work 4

Evidence Supporting the Novelty Claim 5

Contents 6

1 Introduction and Problem Statement 7

2 Background and Related Work 8

3 Requirements and Development Methodology 9

4 System Architecture 10

5 Operating Systems Algorithms 11

6 Storage Shell and Observability 14

7 SpillGuard Native Implementation 16

8 Experimental Framework and Evaluation Method 18

9 Software Testing and Error Handling 20

10 Results Performance and Discussion 22

11 User Manual and Demonstration Procedure 26

12 Conclusion and Future Enhancements 28

References 29

1 Introduction and Problem Statement

1.1 Practical problem

Operating systems teaching frequently separates scheduling calculations, paging exercises and deadlock matrices from the behavior visible in a running application. Students can compute a safe sequence without seeing how assumptions about resource release affect a real long-running service. Similarly, a user observing a slow local AI workload may see high GPU-memory usage without knowing whether the symptom is intentional CPU offload, memory pressure, or a reproducible performance failure. These are related problems of observability and explanation, but they require different evidence.

The project addresses both through a shared desktop: an inspectable simulation for fundamental concepts and a native telemetry path for GPU-memory experiments. The simulator makes state transitions visible and controllable. SpillGuard supplies observations from the actual host and experimental records whose workload, pressure schedule and recovery can be inspected. The problem statement is therefore to build a modular operating systems laboratory that demonstrates core algorithms and supports evidence-based diagnosis of local GPU-memory performance problems. [1, 3–8]

1.2 Objectives

Table 1 Project objectives and acceptance evidence

ID	Objective	Acceptance evidence
O1	Expose process state and two-core dispatch	Executable process table, affinity controls and Gantt history
O2	Implement memory and resource algorithms	Frame allocation, LRU/FIFO/CLOCK, Banker safety and wait-for cycles
O3	Demonstrate storage and shell operations	VFS operations, simulated crash recovery and shell commands
O4	Connect OS concepts to measured host behavior	Collector, replay, fit estimation and admission explanations
O5	Support reproducible evaluation	Matched trials, tri-state verdicts, manifests and retained controls
O6	Deliver a demonstrable modular application	Passing software checks, operating manual and feature walkthrough

1.3 Scope and boundaries

The implemented platform is a React application with a TypeScript simulation kernel and a Python Windows agent. CPU cores, processes, privilege rings, page faults and storage crashes inside the browser are simulated. The native agent observes the Windows host and starts only its own bounded experimental helpers. The project does not replace Windows, implement a bootloader, provide a hardware memory-protection boundary, or load Linux eBPF programs.

The project integrates algorithm visualization, native GPU-memory observation and controlled performance experiments. Its distinctive feature is the connection between inspectable OS concepts and a recorded real-application slowdown. Scheduling, Banker safety and delta debugging remain established algorithms; the empirical claim concerns the measured local workload and the interventions evaluated here.

2 Background and Related Work

The conceptual organization follows virtualization, concurrency and persistence [1]. Processes expose CPU allocation, frame and resource models expose capacity constraints, and inodes illustrate naming and storage. Intermediate states remain visible to the learner.

2.1 Teaching systems and algorithm simulators

MIT xv6 is a complete teaching operating system with a real kernel [2]. Aethor OS runs inside a browser and prioritizes visualization and rapid demonstration. A simulator exposes queues and matrices with little setup; xv6 provides actual traps, page tables, system calls and kernel concurrency. No controlled learning comparison was performed.

2.2 Native GPU-memory observation

DXGI distinguishes an application budget from its current usage and describes penalties from exceeding that budget [3]. NVIDIA documents system-memory fallback when GPU memory is exhausted [4]. SpillGuard distinguishes physical capacity, observer budget, committed memory and shared counters. These observations indicate allocation and pressure rather than tracing every page migration.

2.3 Reduction and experimental diagnosis

Delta debugging isolates failure-inducing circumstances by testing smaller inputs [5]. SpillGuard Repro searches action deletion, allocation size, duration and overlap within explicit budgets. Reduction is established prior art. The phase-aware variant preserves measured proxy signatures; preservation of a verified residency mechanism remains unestablished. [10, 12]

Table 2 Relationship to relevant approaches

Approach	Established role	Role in this project
Operating systems textbook and exercises [1]	Explain policies and invariants	Algorithm background and learning vocabulary
xv6 [2]	Study a working teaching kernel	Boundary reference for real kernel versus simulation
DXGI and vendor memory guidance [3, 4]	Describe memory budgets and fallback	Interpretation of host telemetry
Delta debugging [5]	Reduce a failing input through experiments	Baseline for action-level scenario reduction
Retained project experiments [8–11]	Local measurements and engineering records	Evidence for scoped performance conclusions

2.4 Distinctive integration

The integration links simulated scheduling, paging and resource-allocation decisions with native memory observations and controlled experiments in one interface. Offline replay makes recorded host behavior available on a lab computer without compatible GPU hardware. Diagnostic accuracy, reducer efficiency and educational effectiveness remain separate evaluation questions; the present results establish the local behavior reported in Chapter 10.

3 Requirements and Development Methodology

3.1 Functional requirements

Table 3 Functional requirements and implementation routes

Requirement	Implementation	Observable output
FR1 Process creation and termination	Kernel.ts and useKernelStore.ts	PID, state, burst, priority, affinity
FR2 Scheduling and two-core execution	Scheduler.ts and Kernel.step	Per-core Gantt and utilization
FR3 Frame allocation and replacement	MemoryManager.ts	Frame ownership and fault count
FR4 Resource safety and deadlock	BankersAlgorithm.ts; DeadlockEngine.ts	Safe sequence or wait-for cycle
FR5 Shell and virtual storage	TerminalApp.tsx; VFS.ts	Built-in commands, inodes and journal
FR6 Event observability	Kernel probes and BPFStudioApp.tsx	Hook counts and bounded events
FR7 Host telemetry and replay	collector.py; HostBridge; useSpillStore	Live or recorded process memory frames
FR8 Advice and evidence inspection	gguf_fit.py; bankers.py; ReproPanel	Fit verdict, report and trial outcomes

3.2 Quality requirements

Quality requirements are inspectable intermediate state, explicit simulation boundaries, bounded histories, predictable replay and conservative error classification. The teaching path runs independently of optional host services. Native experiments retain invalid trials and missing blocks, while the local service checks its session token and loopback origin. Security checks implement the intended local access model; comprehensive security auditing is outside this evaluation.

3.3 Methodology

Development proceeds through five stages: define the typed data structures and algorithm modules; coordinate discrete execution ticks; expose state and controls through the desktop; integrate the native telemetry and replay path; and evaluate unchanged workloads with controlled pressure and recovery. Separate simulation and host-state stores maintain the boundary between educational examples and hardware observations.

Verification separates algorithm assertions, application build checks and hardware experiments. Unit tests check specified software behavior; compilation checks type consistency and packaging. The dated experiment records support the performance conclusions in Chapter 10, including negative controls and inconclusive attempts.

3.4 Tools and implementation responsibility

The frontend uses TypeScript, React 18 and Vite; Zustand manages shared state, react-rnd provides window geometry, and Tailwind CSS supplies styling. The native component uses Python 3.12, FastAPI and uvicorn, with pywin32, ctypes, NVML bindings, psutil and httpx for collection and integration. GGUF metadata supports model estimation. Vitest and pytest provide automated tests, while Git records source history. Varad Patil is responsible for the individual project implementation and demonstration. [6, 7]

4 System Architecture

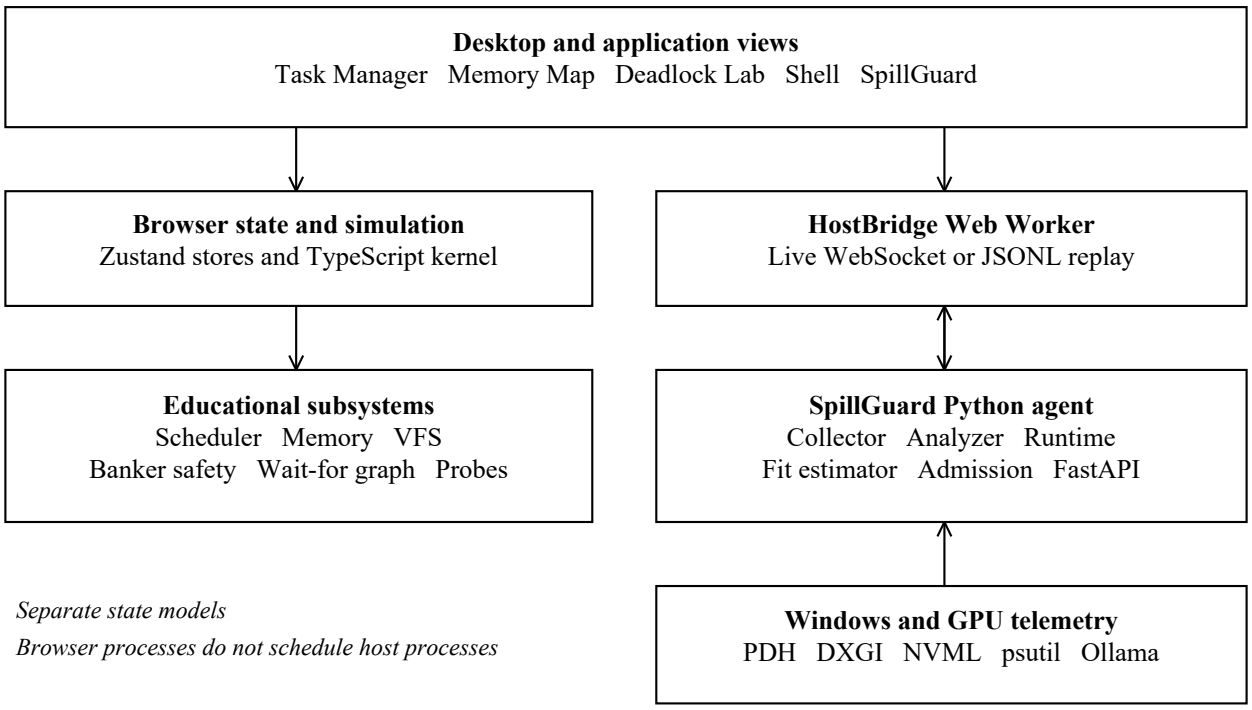


Figure 1 Layered architecture and the boundary between simulation and host observation

4.1 Presentation and state

The desktop imports 18 registered window applications. Window state records position, size, z-index and open, minimized and maximized flags. The frame component supplies dragging and resizing, while the window store clamps its vertical position below the menu. The active-user and boot screens are presentation features rather than host authentication. Supporting music, calculator, messages, community and companion features enrich the desktop experience but do not implement native operating systems services. [6, 7]

useKernelStore publishes process arrays, frame state, per-core execution histories, VFS metadata, probes, fault information and statistics after kernel updates. useSpillStore separately stores the latest host frame, a 300-frame history, events, replay controls and observed memory peaks. This separation prevents simulation PIDs and logical ticks from being mistaken for Windows processes and wall-clock measurements.

4.2 Host data path

The Python collector obtains per-process and adapter counters, the analyzer annotates samples, and the runtime records and publishes them through FastAPI. HostBridge owns WebSocket and replay behavior in a Web Worker, keeping transport and JSONL processing off the main rendering path. The same frame schema supports online inspection and offline demonstration. The browser replay path consumes analyzed traces; raw samples must first pass through the analyzer. [7, 8]

4.3 Persistence model

No relational database is required by the implemented design. The simulator VFS and execution histories are in-memory structures. Native traces use JSONL; research recipes, reports and manifests use JSON; metadata and calibration use structured files. A downloaded disk snapshot is an export, not automatic VFS persistence across sessions. The session token is retained in browser session storage for reconnection. These storage mechanisms have distinct lifetimes and should not be described as one persistent database.

5 Operating Systems Algorithms

5.1 Process lifecycle and dispatch

A process control block (PCB) records each simulated process and its PID, name, READY/RUNNING/WAITING/TERMINATED state, priority, burst, remaining ticks, arrival and first-run ticks, completion, waiting time, memory demand, frame IDs, locks and core affinity. PIDs start at 100. The kernel bootstraps four model processes. Its default wall-clock interval is 800 ms, but the logical step method can be exercised directly by tests. Each tick permits at most one process on each of two model cores. [6]

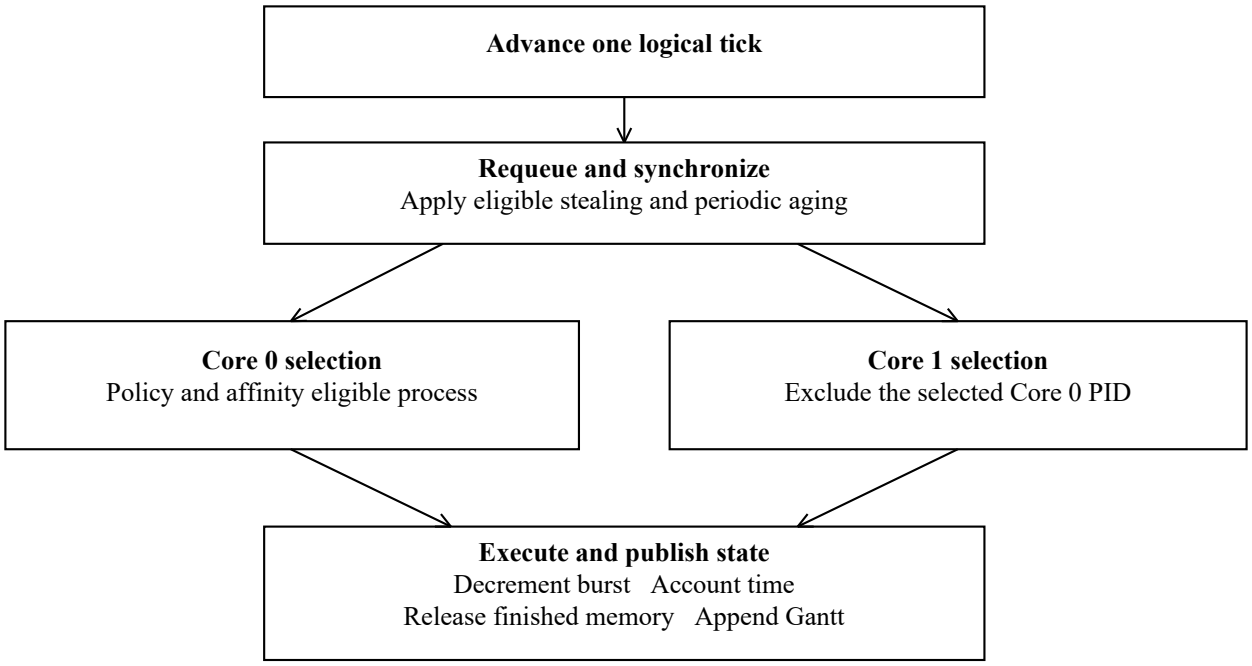


Figure 2 One kernel tick and two-core process selection

At the start of a tick, previously RUNNING processes return to READY. Scheduler synchronization removes stale entries and inserts eligible processes into queues. Idle-core work stealing considers the other queue only when it contains more than one task, scanning its tail and respecting affinity. The second selection excludes the PID chosen for the first core. Execution consumes one burst tick, updates first response and completion fields, touches allocated frames and publishes a bounded Gantt history.

5.2 Scheduling policies and actual semantics

Table 4 Scheduling policies in the current dispatcher

Policy	Selection rule	Important boundary
FCFS	Earliest arrival among eligible core candidates	Selection is reconsidered every tick
SJF	Smallest total burst among candidates	No separate running-process retention rule
SRTF	Smallest remaining burst	Tick-level preemption is modeled
PRIORITY	Lowest priority number, then arrival	Priority is constrained to 1–5 on spawn
ROUND_ROBIN	Core queue head until quantum rotation	Default quantum is four ticks
MLFQ	Core queue order with level-dependent rotation	Quanta 4/8/16; boost every 40 ticks; core selection does not sort by queue level

FCFS means first-come, first-served; SJF, shortest job first; SRTF, shortest remaining time first; and MLFQ, multilevel feedback queue. The integrated multicore MLFQ dispatcher uses core-queue order with level-dependent quanta; the legacy single-core method checks queue levels in priority order. FCFS and SJF are also reconsidered each tick. The current model therefore demonstrates these policies through simplified multicore selection; strict textbook MLFQ priority and non-preemptive retention require additional rules.

5.3 Time accounting

Waiting ticks increase for READY processes that are not selected. Response time is firstRunTick minus arrivalTick, and turnaround is completionTick minus arrivalTick. Because step increments the clock before first dispatch, a process created at tick zero and first run on the first step records response one. This convention must be preserved when comparing hand calculations with exported fields.

CPU utilization is accumulated busy core-ticks divided by total available core-ticks. The reported throughput multiplies completed-process count per logical tick by 100, so it is interpreted as completions per 100 ticks. It is not host CPU utilization or processes per second. Killed processes enter the terminated population without all normal completion metrics, so averages that mix killed and normally completed processes require care.

5.4 Memory allocation and replacement

MemoryManager creates 32 physical frame records by default. Each page occupies four kibibytes (KiB), so a process needs ceil(memorySizeKB / 4) frames and modeled physical capacity is 128 KiB. Free frames are assigned first. For a deficit, the fault counter increases by the additional pages needed and occupied frames may be evicted. Least recently used (LRU) orders victims by last access tick; first in, first out (FIFO) orders them by load tick. CLOCK gives referenced frames a second chance through a moving hand.

This is an allocation and replacement model, not an instruction-level demand-paging engine. There is no complete virtual-page reference stream, TLB, swap-device latency or page-table fault handler. The optional OPTIMAL label falls back to LRU because future references are not available. Frame records and process allocation lists also need stronger ownership invariants under eviction; the current touch path indexes frames from a process list without revalidating the owning PID. A failed allocation may leave partial assignments, and the spawn path does not reject the new process based on the allocation result.

The privilege demonstration reserves logical addresses 0x0000–0x3FFF for kernel space and allows user-space addresses through 0xFFFF. A simulated Ring 3 violation creates a register dump and places the

process in WAITING. The 64 KiB logical address range and the 128 KiB frame pool are separate teaching abstractions rather than a validated coherent hardware address-translation system.

5.5 Resource allocation safety

The Banker module computes Need as Maximum minus Allocation, copies Available into Work, and finds unfinished processes whose Need does not exceed Work in any resource dimension. Simulated completion adds each process allocation back to Work. A complete sequence establishes safety under the declared claims and completion assumptions. An unsafe result means no complete safe sequence was found; it does not itself prove an existing deadlock. The function validates matrix dimensions but requires callers to supply non-negative, consistent claims with Maximum at least Allocation. [6]

Table 5 Worked Banker safety example

Process	Allocation A B	Maximum A B	Need A B
P0	1 0	2 1	1 1
P1	0 1	1 2	1 1

For Available = (1, 1), P0 can complete, producing Work = (2, 1); P1 can then complete, producing Work = (2, 2). The sequence P0, P1 is safe. This is a worked illustrative input, not a hardware measurement. Its purpose is to show how each allocation is returned only when completion is permitted.

5.6 Wait-for deadlock detection

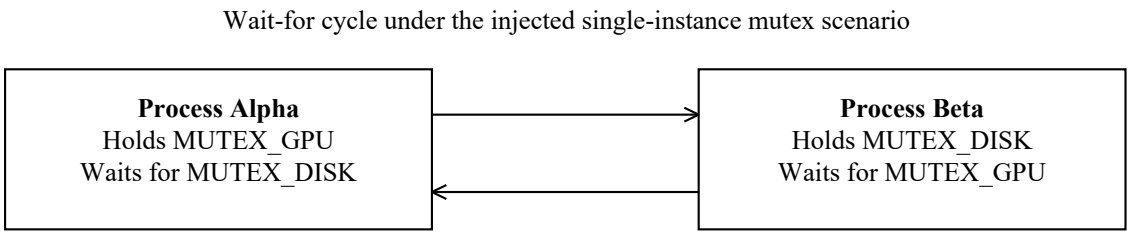


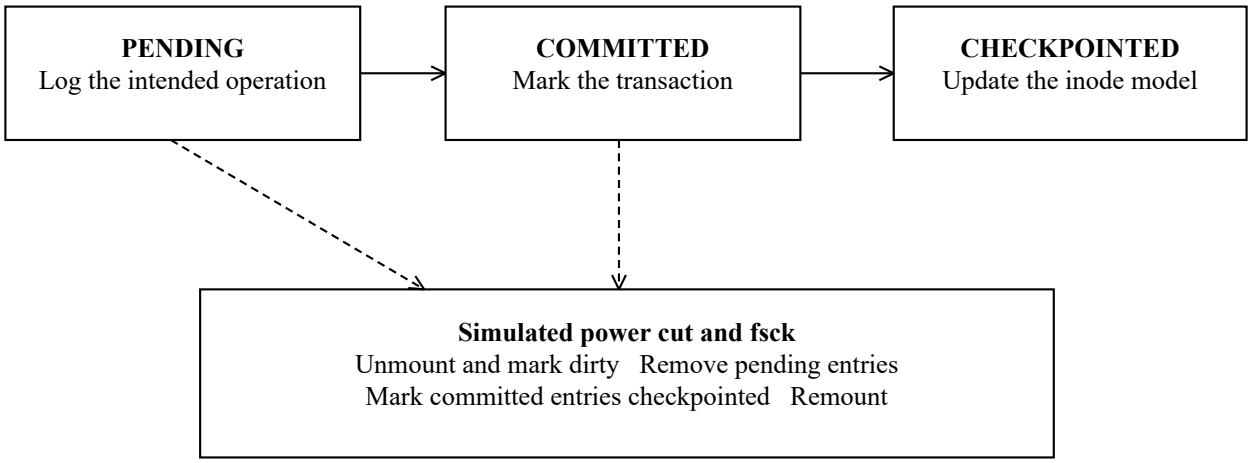
Figure 3 Injected circular wait between two simulated mutex holders

DeadlockEngine builds waiter-to-holder edges from locksHeld and waitingFor, then runs depth-first search with a recursion stack to find a cycle. It reports the first detected cycle. The injected two-mutex case is a suitable single-instance demonstration. General multi-instance resource deadlock detection would require counts and a resource-allocation analysis beyond the current graph model; the separate Banker module addresses declared maximum claims rather than converting every graph cycle into a universal deadlock result.

6 Storage Shell and Observability

6.1 Virtual file system

VFS represents files and directories as inode records containing an inode number, name, type, size, permissions, parent ID and content. It initializes root, user and etc directories, plus sample files. Current-directory traversal supports named child directories, parent traversal, root and home shortcuts. File creation, directory creation and removal check the mounted flag and same-directory name collisions. The permission string is displayed metadata rather than an enforced host access-control policy. [6]



No durable block device or general operation-payload redo is implemented.

Figure 4 Journal states and the implemented simulated crash-recovery path

Normal updates move through PENDING, COMMITTED and CHECKPOINTED states in one synchronous operation. triggerPowerCut marks the superblock dirty, unmounts the model and appends a pending torn-write entry. recoverFSCK removes pending entries, marks committed entries checkpointed and remounts. It does not reconstruct a durable inode table from block storage or generally reapply operation payloads. The exercise demonstrates transaction states and rollback reasoning, not validated disk crash consistency.

The journal is bounded at 25 entries. Removing a node filters the node and its immediate children, so arbitrary-depth recursive deletion is not fully implemented. Some file sizes are display approximations rather than encoded-byte counts. These boundaries matter when presenting the VFS as an educational model and when choosing future tests.

6.2 Shell interface

aethersh routes built-in commands to the simulation store and SpillGuard views. Process commands include ps, run, kill, sched, quantum and affinity. Memory and diagnostic commands include memstat, gpf and bpf. Storage commands include ls, pwd, cd, cat, touch, mkdir, rm, powercut and fsck. SpillGuard commands expose status, explanations, events, a fit request and demo replay. The shell does not parse general Unix pipelines, execute arbitrary host binaries or implement POSIX job control.

6.3 Event probes

The kernel provides hook categories and named probes with enabled flags, event counts and map outputs. emitBPFEvent appends to a bounded event list and increments matching probe maps. The stored filterScript describes the requested probe but is not an executed eBPF program or an arbitrary filter interpreter. The studio is therefore an eBPF-inspired observability model. Simulator page-fault events include memory mapping and injected GPF notifications; their event counts are not identical to MemoryManager.totalPageFaults.

6.4 Application module inventory

Table 6 Registered desktop applications and responsibilities

Module	**Role**
Task Manager	Spawn, terminate and inspect processes; schedule and affinity controls
Memory Map	Physical-frame view and host GPU-memory twin view
Deadlock Lab	Banker matrix, wait-for graph and live/replay VRAM admission
Terminal	Built-in simulation and SpillGuard command interface
Finder and Trash	Virtual inode browsing and desktop deletion presentation
Kernel Monitor and Test Runner	Statistics, fault dumps, audit journal, tests and snapshot export
eBPF Studio and KernelQuest	Event probes and guided learning scenarios
SpillGuard	Live/replay memory, fit advice and research evidence desk
About, Settings, Internet	Project information, visual settings and simulated browser
Music, Calculator, Messages, Community	Supporting desktop interactions; application-level behavior

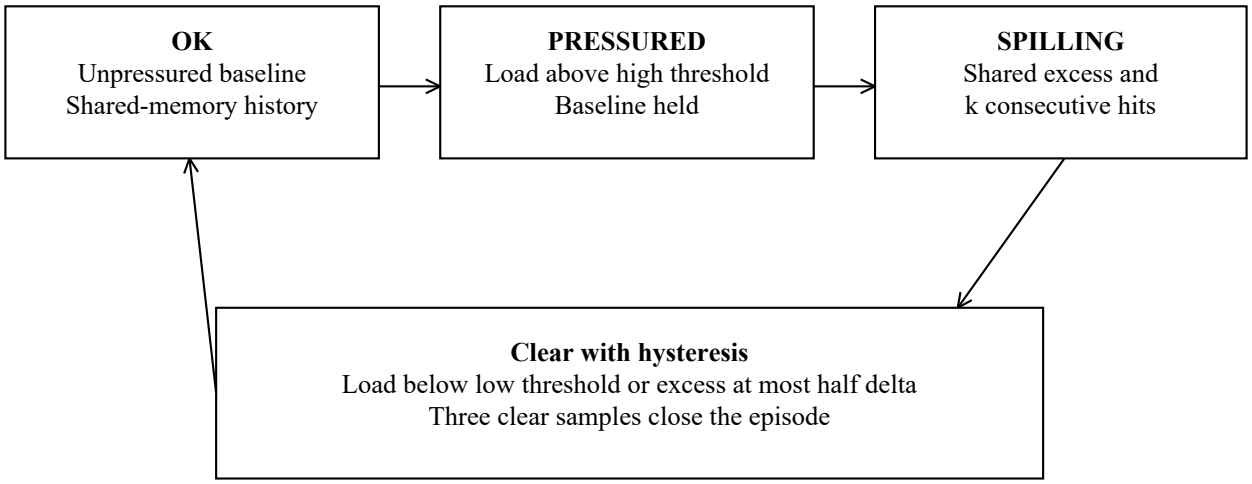
7 SpillGuard Native Implementation

7.1 Collector and observation semantics

The collector enumerates Windows Performance Data Helper (PDH) counter instances, extracts process identifiers and locally unique adapter identifiers (LUIDs), and aggregates one selected adapter. DirectX Graphics Infrastructure (DXGI) supplies capacity and the calling observer budget; NVIDIA Management Library (NVML) can supply device totals. psutil supplies process and host-memory information, and Ollama polling identifies framework-reported model allocation. Missing optional sources remain unavailable rather than fabricated. Process creation time distinguishes a reused PID from the original process. [7, 8]

Fields ending in _mb use mebibytes (MiB), where one MiB is 1,048,576 bytes. PDH dedicated usage, shared usage and committed usage measure different quantities. Dedicated usage concerns the local GPU-memory segment, shared usage concerns system-memory-backed GPU allocations, and committed usage describes committed demand. These counters do not expose each page migration; shared usage alone cannot establish a severe slowdown.

7.2 Detector and classifier



An alert is a memory-pressure indicator; severe slowdown needs a matched experiment.

Figure 5 Memory-pressure detector with entry and exit hysteresis

The detector computes adapter load as dedicated usage divided by physical VRAM. Defaults are a high threshold of 0.92, low threshold of 0.85, shared excess of more than 128 MiB, two consecutive entry hits and three clear samples for exit. A process baseline is the minimum shared usage observed during its retained unpressured history; initial observation is a fallback when no such history exists. Shared excess and recent growth support entry. Hysteresis reduces transitions caused by a single fluctuating sample.

The analyzer distinguishes framework-reported CPU offload (A), its memory-spill heuristic (B), and host RAM pressure (C). Combined B+C events indicate overlapping conditions. Culprit ranking selects the process with the greatest dedicated-memory growth in the preceding window, using held memory as a tie-breaker. This is an explanation heuristic, not proof that the selected process caused a paging mechanism. [8]

7.3 Fit estimation

The estimator decomposes the GPU footprint into weights, the key–value (KV) cache, compute buffers and runtime overhead. GGUF, the model file format used by the supported local inference workflow, supplies tensor metadata for layer and auxiliary weights. KV demand depends on GPU layers, context length, KV heads, head dimensions, element format and parallel sequences. The implementation handles f16 and quantized KV formats and relevant vision weights and buffers. Machine-specific calibration is stored in agent/calibration.json. [7, 8]

For an architecture with equal key and value head dimensions, the explanatory relationship is KV bytes = 2 × GPU layers × context tokens × KV heads × head dimension × bytes per element × parallel sequences. The actual implementation uses available model metadata. This estimate is advisory: it does not reserve host VRAM or prevent a competing process from allocating after the check.

7.4 Admission for long-lived processes

The native Banker-style model tracks VRAM and commit resources. Long-lived jobs reserve their remaining maximum claim up front, because a browser or service cannot be assumed to finish and release its allocation during a new launch. The classic search then applies to finite jobs. The browser comparison uses a single VRAM resource with a 256 MiB margin and observed peak multiplier 1.1. It shows how a textbook safe sequence and a long-lived reservation can disagree.

The adviser searches configurations such as context length, KV type, flash attention and offloaded layers. A FIXABLE result means a modeled configuration meets its checks; it is not an externally enforced guarantee. Maximum demand inferred from past peaks can underestimate future demand, and a prelaunch check can become stale. The native launch workflow is CLI-only and requests interactive confirmation; the dashboard API provides observation and advice.

7.5 Transport and local service boundary

FastAPI checks the Host and Origin headers and compares a session token for protected HTTP and WebSocket paths. The served page receives the token through a URL fragment and removes it from the address bar after reading it. WebSocket messages then carry the analyzed frames. A bounded queue drops the oldest pending frame for a slow client instead of growing without limit. These checks support the intended loopback workflow; they do not imply protection against arbitrary local code or a compromised browser context.

8 Experimental Framework and Evaluation Method

8.1 Research questions

Table 7 Research questions and current evidence

Question	Evidence available	Status
RQ1 Can controlled memory pressure reproduce a real application slowdown?	October 18 dose response and fresh witness validation	Supported locally for one fixed Qwen2.5 workload
RQ2 Can smaller scenarios preserve the relevant failure?	Plain and phase-aware reducer code; software fixtures	Implemented; live-witness reduction benefit not established
RQ3 Can an owned intervention restore target performance without changing work?	Five remedy conditions with matched trials	Supported in the saved one-machine pilot
RQ4 Do counter alerts identify harmful pressure reliably?	Nine valid dose/idle trials and retained W1 traces	Scope-dependent; harmfulness cannot be inferred from counters alone

8.2 Matched trial protocol

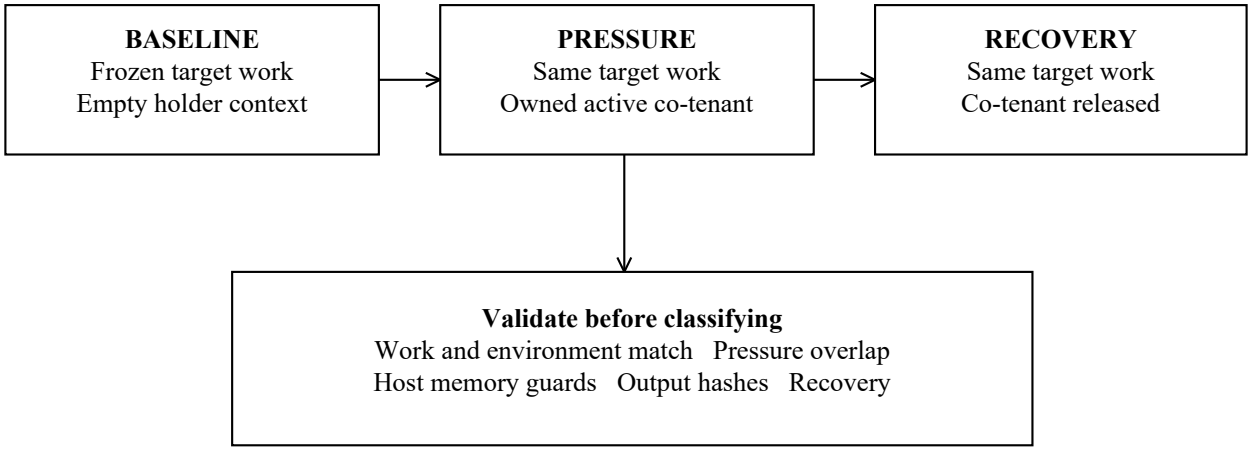


Figure 6 Baseline–pressure–recovery protocol and conservative verdicts

A recipe freezes the target, number of work units, protocol bounds, repetitions, preparation mode and pressure actions. An action defines allocated bytes, timing, duration and touch interval. Fixed baseline, pressure and recovery stages run in order. A pressure manipulation must be observed while the target executes. For inference, model identity, context, prompt, generated output and thinking hashes are checked; partial CPU offload or changed work makes the result inconclusive. [9, 10, 12]

The primary inference metric is server-reported decode milliseconds per token. The slowdown ratio is pressure metric divided by baseline metric; recovery is recovery metric divided by baseline metric. FAIL means the declared slowdown threshold was reproduced after the validity and recovery checks. PASS means a valid trial remained below that threshold. An error, missing observation, changed environment,

guard stop or mixed repeated outcomes remains INCONCLUSIVE. FAIL is therefore the experimental target outcome, not a failing software unit test.

8.3 Evidence bundles and bounded execution

The runner stores a recipe, report, readable summary, stage observations, helper events, errors and a SHA-256 manifest. Bundle verification checks paths, expected files and hashes. A hash manifest detects changes relative to the recorded manifest; it is not an authenticity signature. Local evidence may include process names and paths, so publication exports require review. The browser evidence desk parses a report but does not itself verify the complete bundle manifest.

Only owned experiment helpers may be stopped. Host-memory floors and commit limits abort unsafe or confounded runs, and cleanup failures invalidate the stage. Default protocol limits and the October 18 pilot settings differ; the report identifies the actual pilot rather than presenting one universal configuration. No new pressure-generating hardware experiment was run during preparation of this report; empirical numbers come from retained records.

8.4 Scenario reduction

The plain baseline attempts action deletion while accepting only reproduced failures. The extended search supports phase-aware, greedy, ddmin, ProbDD-style, no-reduction and tiny exhaustive methods, with explicit candidate and execution budgets. Fresh validation is reserved separately from search. The phase-aware method compares observed proxy signatures and can reject a mechanism-class switch, but those signatures are not direct residency ground truth. A locally minimal result under specified transforms is not a global minimum, and fixture success is not empirical superiority on the real witness.

9 Software Testing and Error Handling

9.1 Software verification results

Software verification on 22 October 2026 completed 66 frontend tests in 12 files and 149 Python agent tests. TypeScript compilation and the Vite production build succeeded, transforming 1,544 modules. These results validate the recorded software assertions and build process. Hardware performance results are evaluated separately using the retained matched trials. [13]

Table 8 Current automated verification results

Check	Command or location	Observed result
Frontend suite	npm test -- --run	66 passed; 12 test files
Production build	npm run build	TypeScript and Vite succeeded
Agent suite	From agent: .venv/Scripts/python.exe -m pytest -q --basetemp <new writable directory>	149 passed
Build output	dist/assets index JS; CSS; worker	474.19 kB JS; 64.80 kB CSS; 2.37 kB worker, before gzip

The agent tests run from the agent directory so pytest resolves the local spillguard package and configuration. The successful verification used a new writable temporary directory through --basetemp. No production-source modification was required to complete the checks.

9.2 Representative test cases

Table 9 Representative executable test cases and results

ID	Input or trigger	Expected behavior	Result
T1	RR, quantum 2; A/B/C pinned to core 0	First eight names A A B B C C A A	Passed
T2	One process with burst 5	Runs for five ticks and terminates	Passed
T3	MLFQ two runnable processes	Initial level-0 quantum then rotation	Passed
T4	Memory and replacement scenarios	Expected allocation/replacement assertions	4 tests passed
T5	Well-formed Banker scenarios	Need and safe/unsafe outcomes	4 tests passed
T6	Wait-for graph scenarios	Cycle and non-cycle classification	3 tests passed
T7	VFS operations and recovery	Directory/file and journal expectations	3 tests passed
T8	Spill admission fixtures	Classic and long-lived verdicts	3 tests passed
T9	Research report and replay fixtures	Schema validation and bundled data parsing	Frontend suites passed
T10	Recipe, evidence and cleanup failures	Conservative verdicts and bundle integrity	Agent suites passed

The test suites cover their stated assertions; they do not remove the documented scheduler, memory-ownership, VFS-persistence and validation limitations. The interface screenshot in Chapter 10

records an earlier evidence-desk checkpoint. Current GUI interaction and live hardware operation should be evaluated separately from unit-test success.

9.3 Error-handling design

Table 10 Error conditions and implemented responses

Condition	Response	User or evidence effect
Wrong service token	HTTP 401; WebSocket closes with 4401	Reconnect using the current agent session link
Unapproved Host or Origin	HTTP or handshake rejection	Prevents cross-origin use in intended local model
Disconnected WebSocket	Bounded increasing retry delay	Visible connection state
Raw or malformed replay trace	Worker reports an error	Analyze raw trace or choose valid JSONL
Malformed research report	Schema and size checks reject it	Visible import failure
Host-memory or commit guard	Stop trial and retain record	INCONCLUSIVE rather than a false result
Changed workload or failed recovery	Invalidate matched trial	Failure is not attributed to the manipulation
Bundle tampering or unsafe manifest path	Verification raises an error	Do not rely on the altered bundle
Unmounted VFS	Reject normal file operations	Run simulated fsck to remount

9.4 Additional regression priorities

Future tests should add integrated policy traces for strict non-preemptive FCFS/SJF, MLFQ priority across core queues, affinity changes after enqueue, eviction ownership consistency, rejected oversized allocation rollback, nested-directory deletion and actual committed-operation redo. UI accessibility and full browser workflows need separate validation. These are identified improvement areas, not fabricated failing test results.

10 Results Performance and Discussion

10.1 Hardware and workload context

Table 11 Environment recorded for the October 18 pilot

Component	Recorded configuration
GPU	NVIDIA GeForce RTX 3050 6 GB Laptop GPU; recorded physical capacity 6,002 MiB
Operating system and driver	Windows 11 build 26200; NVIDIA 581.83
CPU and RAM	AMD Ryzen 7 7840HS; 16 logical threads; 16 GB system RAM
Inference runtime	Private Ollama 0.34.4 server; qwen2.5:3b-instruct
Frozen work	Context 2,048; 128 generated tokens; temperature zero; fixed prompt
Pressure process	Owned CUDA holder; active touch interval 250 ms
Design	Two matched trials per active dose level; baseline, pressure and recovery

The recorded capacity is a counter-derived MiB value, while the product name says 6 GB. The dose recipe estimates capacity headroom from physical capacity minus desktop residency and target footprint, then adds a planned overflow. Planned overflow is an experimental setting, not a direct measurement of bytes currently resident outside VRAM. [9, 11]

10.2 Dose response

Table 12 Active co-tenant dose response with individual trials

Overflow MiB	Holder MiB	Baseline ms/token	Pressure ms/token	Ratio	Verdict
-512	2941	14.42 / 14.28	16.63 / 16.67	1.15 / 1.17	PASS / PASS
0	3453	14.20 / 14.00	17.11 / 17.12	1.20 / 1.22	PASS / PASS
256	3709	14.04 / 14.06	33.31 / 33.07	2.37 / 2.35	FAIL / FAIL
512	3965	14.15 / 14.10	33.61 / 32.18	2.38 / 2.28	FAIL / FAIL

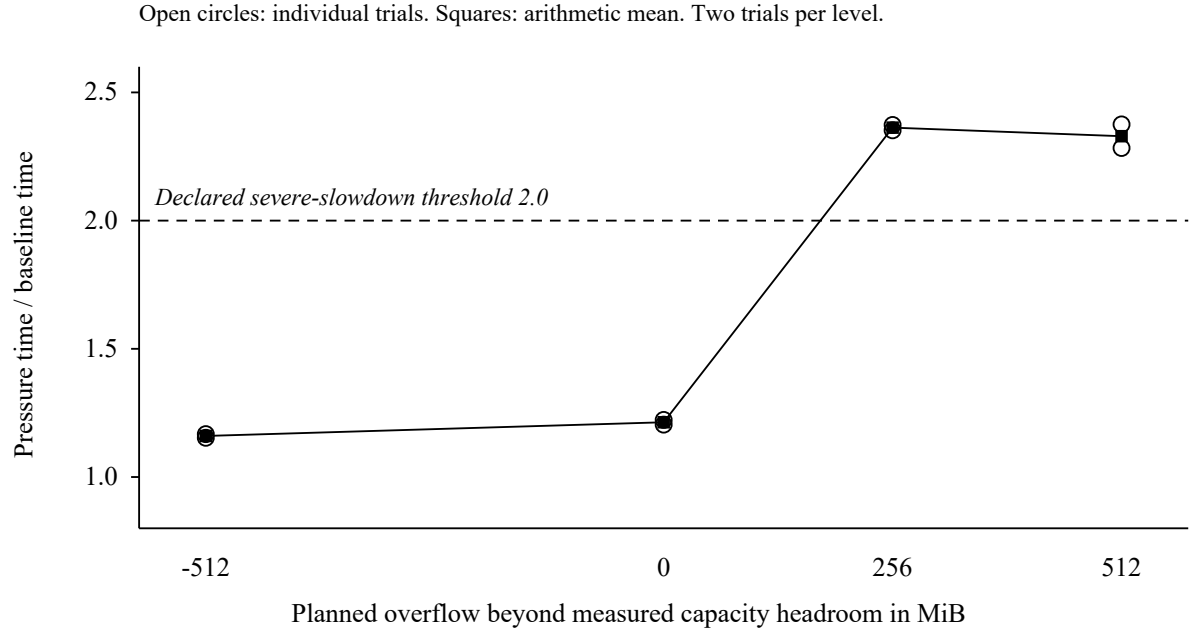


Figure 7 Recorded performance cliff near the planned capacity boundary

The two below-capacity controls were approximately 1.15–1.17 times baseline. At capacity, ratios were approximately 1.20–1.22. At +256 MiB, they rose to 2.35–2.37; +512 MiB produced 2.28–2.38. The response therefore shows a local cliff followed by a plateau over the tested levels, rather than a linear relation between requested overflow and slowdown. These are descriptive pilot observations with only two trials per level; no statistical-significance or population-generalization claim follows.

10.3 Activity intervention and detector limitations

The retained analysis compares an active +256 MiB holder with an idle holder that allocated and touched once. The reported process and adapter memory counters were the same at the displayed resolution: the target held 2,033 MiB dedicated and 388 MiB shared, and the adapter was near its recorded ceiling. The active case slowed approximately 2.37 times, while the valid idle case was approximately 1.03 times baseline. Two other idle attempts were stopped by the host-commit guard and remain inconclusive. [9]

Across nine valid dose/idle trials, the original counter detector alerted on all four severe failures and on three of five non-failures: trial-level recall 4/4 and precision 4/7. This does not contradict its earlier W1 pressure-detection results; the labels and target question differ. Detecting shared-memory growth is not equivalent to detecting harmful slowdown. An active holder also introduces bandwidth and compute interference. Repeated page migration remains a hypothesis pending validated residency-event evidence.

10.4 Remedy evaluation

Table 13 Recorded interventions on the +256 MiB witness

Intervention	Holder remaining MiB	Individual ratios	Outcome
Fresh original witness	3,709	2.35 / 2.34	FAIL / FAIL
Release entire owned holder	0	1.02 / 1.00	PASS / PASS
Serialize background after target	0 during target	1.00 / 1.00	PASS / PASS
Release half the held bytes	1,854	1.10 / 1.10	PASS / PASS
Release one eighth	3,245	1.19 / 1.20	PASS / PASS
Pause activity and keep allocation	3,709	1.01 / 1.01	PASS / PASS

The evidence summary reports six reproduced active witness trials across the initial doses and fresh validation, with ratios 2.28–2.38. Pausing activity while keeping the allocation restores the target near baseline in the retained two-trial remedy condition. Serialization also finishes the owned background work after the target. These outcomes support an owned-workload mitigation workflow; they do not authorize or validate automatically pausing unrelated third-party applications.

10.5 Earlier calibration and synthetic results

Table 14 Earlier evaluation and its scope

Experiment	Recorded result	Interpretation
W1 block-write bandwidth, n = 3	Resident 163.3 ± 0.1 GB/s; slow blocks 13.2 ± 0.0 GB/s	About 12.4 times bandwidth difference in this synthetic test
W1 detector, 3 spill and 2 near-full controls	3/3 detected; zero alerts on two negative runs; latency 0.95 ± 0.03 s	Small historical trace set, not ordinary-session accuracy
Fit estimator, 9 full-offload configurations	Leave-one-out MAE 46 MiB; maximum 93 MiB	Local calibration; related configurations are not independent machines
Ollama size_vram comparison	Mean absolute difference 902 MiB	Comparison to measured runner footprint in that calibration
Original agent overhead, five runs	2.3 ± 0.3% of one core; approximately 52 MiB RSS	About 0.14% of 16-thread aggregate CPU; not full framework overhead

The ± values are means and standard deviations retained from the earlier evaluation, not confidence intervals. The slow-block standard deviation rounds to 0.0 at the displayed precision. Mean absolute error (MAE) is the average absolute prediction difference; leave-one-out validation fits the estimator with each configuration omitted in turn. RSS is resident set size. The original 1 Hz agent overhead does not include every later experiment-orchestration cost. [8, 10, 11]

10.6 Interface evidence

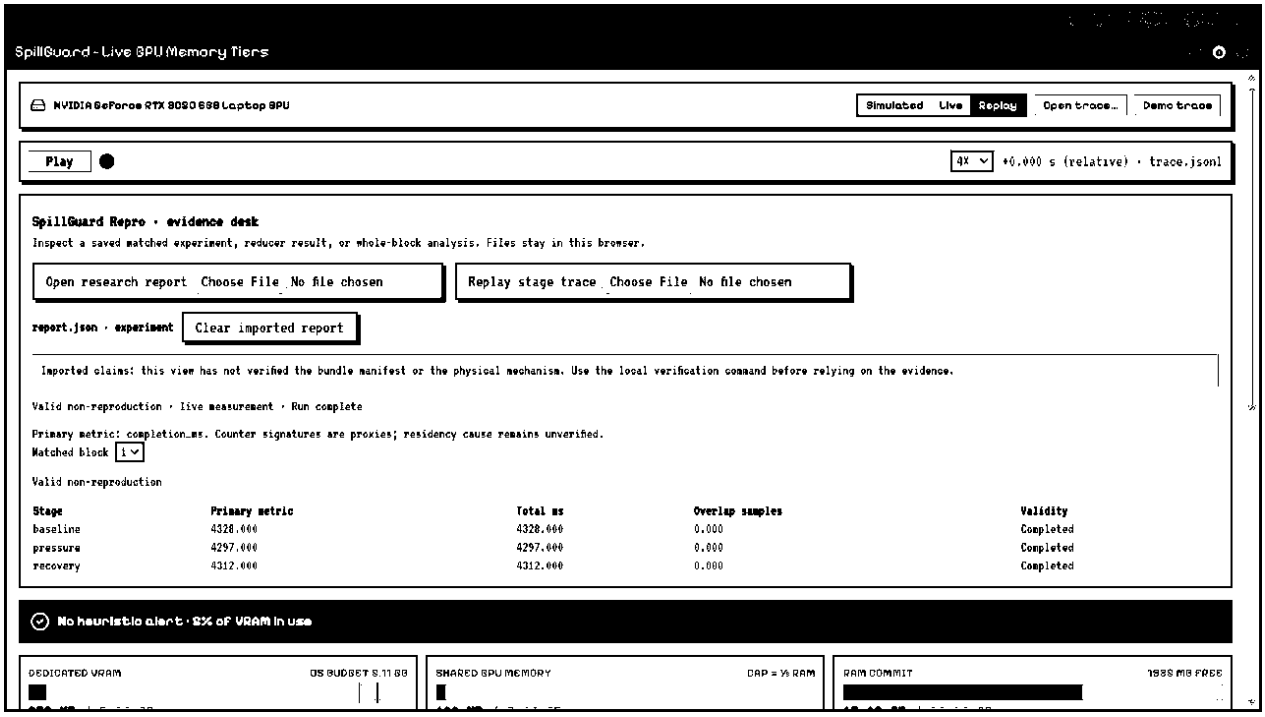


Figure 8 Retained October 16–17 evidence-desk screenshot rendered in black and white

This retained interface checkpoint displays a valid native CUDA negative control and relative-time replay. It is included to document the evidence-inspection implementation. It does not depict the October 18 positive inference witness. The current research library contains separately labeled dose-response, remedy, earlier analysis, fixture and replay inputs, avoiding presentation of a fixture as a hardware result.

10.7 Threats to validity

External validity is limited by one laptop, one GPU vendor and driver, one validated real inference target, a synthetic owned co-tenant, and a small sample. Internal validity is limited by fixed baseline–pressure–recovery order, warm/cold preparation effects, and unmeasured temperature, clocks and fine-grained residency behavior. The idle intervention helps isolate holder activity but does not by itself establish a page-migration mechanism. Construct validity requires care because counter pressure, bandwidth proxies and severe application slowdown use different labels.

The results establish executable software and a one-machine performance witness. Generalization is limited by the absence of an external reproduction, a completed multi-machine campaign, a user study and a validated reduction advantage on the real witness. These limits constrain the interpretation of the measured slowdown and the scope of future claims.

11 User Manual and Demonstration Procedure

11.1 Browser-only startup

```
npm install
npm run dev
```

Open http://localhost:3000. Complete the presentation boot/login flow and enter the desktop. On a machine using an existing dependency installation, the start command can be run directly. For a production package, run npm run build and npm run preview. The preview command reports its actual URL. The browser-only path demonstrates the simulator and replay without the native agent.

11.2 Scheduling demonstration

Open Task Manager and pause the clock. Terminate any existing simulated processes to isolate the example. Select Round Robin, set quantum to two, and create A, B and C in that order, each with a burst of at least six ticks and Core 0 Only affinity. Set a slow clock speed, resume and inspect the first six new Gantt entries: A A B B C C. Existing Gantt entries precede this example. The interface has pause and resume controls; exact tick-by-tick sequences are also checked by the dispatcher tests. These actions affect simulated processes, not Windows processes.

11.3 Memory and deadlock demonstration

Open Memory Map and inspect the 32-frame pool. Spawn processes until allocation requires replacement, then compare LRU, FIFO and CLOCK using controlled inputs. Explain that this view models allocations rather than a complete demand-paging trace. In Deadlock Lab, run a well-formed Banker scenario and read the Need matrix and safe-sequence steps. Trigger the injected circular wait and inspect the two-process graph. Keep the multi-instance safety example distinct from the single-instance wait-for cycle.

11.4 Shell and virtual storage demonstration

```
help
ps
run demo_worker 12
sched ROUND_ROBIN
quantum 2
memstat
mkdir demo
cd demo
touch sample.txt
ls
cd ~
powercut
fsck
```

Use the PID returned by run when demonstrating kill, affinity or gpf; do not assume a fixed PID. During the simulated power cut, show that file operations are unavailable. After fsck, inspect the pending-entry rollback and remount state. Explain the journal simulation boundary before describing recovery.

11.5 Live native startup

```
npm run build
cd agent
.venv/Scripts/python.exe -m spillguard serve
```

If no environment exists, create a Python virtual environment and install agent/requirements.txt first. The retained project environment uses Python 3.12. The service prints a loopback URL containing the current session fragment; open that exact link, then launch SpillGuard. The default service port is 8765. A restarted agent uses a new token, so reopen the new printed link if the interface reports an expired session.

11.6 Replay and research reports

Open SpillGuard, choose Replay and load the demo trace. Use Play, speed and seek controls. An analyzed trace contains a header followed by frames with summary fields. Raw collector output must be processed with spillguard analyze before import. The evidence desk can open research JSON reports, including matched experiments and reductions. Report parsing is not equivalent to verifying the complete local evidence bundle.

Load the measured dose-response entry to inspect individual outcomes. Explain PASS, FAIL and INCONCLUSIVE before presenting the chart. For the retained screenshot, identify it as an earlier negative-control checkpoint. Use Live only when the intended Windows agent and compatible hardware are available; Replay provides an alternative demonstration of recorded behavior.

11.7 Reproduction and integrity commands

```
agent/.venv/Scripts/python.exe bench/repro.py validate bench/recipes/cuda-smoke.json
agent/.venv/Scripts/python.exe bench/repro.py verify <bundle-directory>
```

Run the commands from the repository root, replacing the bracketed value with an existing evidence directory. A new experiment output directory must not already contain an earlier bundle. Live stress and inference commands require the prescribed model, hardware and sufficient host memory. Inspect the stored recipe, preparation mode and limits before attempting to reproduce the hardware pilot.

11.8 Suggested final demonstration order

Table 15 Demonstration script with observable checkpoints

Step	Demonstration	Evidence to explain
1	Desktop and scope boundary	Browser simulation versus native observation
2	Process dispatch and RR quantum	Visible Gantt order and per-core state
3	Memory and resource examples	Frame policy, safe sequence and wait-for cycle
4	Shell and storage crash	Commands, journal states and remount
5	Probes and verification	Event map, test results and export
6	SpillGuard replay and dose response	Recorded work, ratios and detector limitations
7	Remedy comparison and limitations	Unchanged target work, recovery and scoped conclusions

12 Conclusion and Future Enhancements

Aethor OS implements a substantial operating systems laboratory through a browser simulator and a native GPU-memory observation and experimentation path. Its core demonstrable concepts include process management, scheduling, memory allocation and replacement, resource safety, wait-for analysis, virtual storage, shell programming and process monitoring. The separation between simulated state and measured host state allows these concepts to be taught together without pretending that a browser application is a hardware kernel.

Current verification supports an executable implementation: 66 frontend tests, 149 agent tests and a successful production build. Retained hardware records add a more specific empirical result: controlled co-tenant pressure reproduced a severe local inference slowdown, and owned interventions restored performance while preserving target work. Similar counters in active and idle conditions show why an alert should be treated as a diagnostic lead rather than sufficient evidence of harmful residency behavior.

Future work should first improve scheduler fidelity, frame ownership and persistent storage, then expand experimental coverage. Each enhancement below has a concrete validation criterion so that implemented functionality and demonstrated benefit remain distinguishable.

Table 16 Prioritized future work and validation criteria

Priority	Enhancement	Validation criterion
1	Strengthen scheduler and frame-ownership invariants	Integrated traces and ownership checks cover documented edge cases
2	Implement persistent VFS with operation-payload redo	Restart and crash-injection tests verify recoverable committed changes
3	Validate actual residency events	A reproducible PIX/ETW interpretation supports the proposed mechanism
4	Evaluate reduction on a real positive witness	Fresh validation and comparator budgets show retained failure and search cost
5	Expand hardware and target coverage	Predeclared trials include more GPUs, drivers, models and controls
6	Measure educational and usability outcomes	A declared study evaluates comprehension and diagnostic tasks
7	Improve accessibility and end-to-end UI verification	Keyboard, viewport and live/replay flows pass documented checks

The completed project connects OS algorithm inspection with native GPU-memory diagnosis and a reproducible local inference experiment. Further instruction emulation, WebAssembly execution or distributed scheduling would extend its scope. The immediate priorities are stronger simulator invariants and broader validation of the existing experimental workflow.

References

[1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau. Operating Systems Three Easy Pieces. Author-maintained online textbook. https://pages.cs.wisc.edu/~remzi/OSTEP/ . Accessed 22 October 2026.

[2] MIT PDOS. Xv6 a simple Unix-like teaching operating system. Official project and teaching resources. https://pdos.csail.mit.edu/6.828/xv6/ . Accessed 22 October 2026.

[3] Microsoft. DXGI_QUERY_VIDEO_MEMORY_INFO structure and IDXGIAdapter3 QueryVideoMemoryInfo. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/dxgi1_4/ns-dxgi1_4-dxgi_query_video_memory_info . Accessed 22 October 2026.

[4] NVIDIA. System Memory Fallback for Stable Diffusion. Support article 5490. https://nvidia.custhelp.com/app/answers/detail/a_id/5490 . Accessed 22 October 2026.

[5] A. Zeller and R. Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE Transactions on Software Engineering, 28(2), 183–200, 2002. Author publication record: https://www.st.cs.uni-saarland.de/papers/tse2002 . Accessed 22 October 2026.

[6] Aethor OS working source. src/kernel/Kernel.ts, Scheduler.ts, MemoryManager.ts, BankersAlgorithm.ts, DeadlockEngine.ts, VFS.ts and src/types/kernel.ts. Workspace inspected 22 October 2026.

[7] Aethor OS integration source. src/stores/, src/components/, src/spillguard/ and agent/spillguard/. Workspace inspected 22 October 2026.

[8] Project technical record. docs/SPILLGUARD.md; bench/results/w1_eval.json; bench/results/e3_summary.json; agent/calibration.json. Retained local evidence and calibration, inspected 22 October 2026.

[9] Project pilot analysis. docs/SPILLGUARD_REPRO_RESULTS_20261018.md. First reproduced witness, dose response, activity intervention and remedies, 18 October 2026.

[10] Project methodology and scope. docs/SPILLGUARD_REPRO.md; docs/SPILLGUARD_REPRO_OLLAMA.md; docs/research/2026-10-16-spillguard-opportunity/claim-evidence.md. Dated development records, interpreted with the later October 18 results.

[11] Project experiment records. traces/repro/20261018-dose-response/summary.json; traces/repro/20261018-remedy-plus256-r2/; public/research/01-demo-dose-response.json and index.json. Local measured records and curated presentation inputs.

[12] Project experiment implementation. agent/spillguard/repro/evidence.py, runner.py, spec.py, reducer.py, search.py, signature.py and workflows.py; bench/dose_response.py. Workspace inspected 22 October 2026.

[13] Verification observed during report preparation. Vitest: 66 passed in 12 files; Python agent: 149 passed; TypeScript and Vite build: successful. Commands and environment handling recorded in Chapter 9.

[14] N. Alarcon. NVIDIA Releases Nsight Systems 2020.2. NVIDIA, 24 March 2020. Windows paging queues, eviction events and video-memory graphs. https://developer.nvidia.com/blog/nvidia-announces-nsight-systems-2020-2/ . Accessed 24 October 2026.

[15] hypomnesis maintainers. hypomnesis::spill module documentation. WDDM saturation and shared-growth detection with spill episodes. https://docs.rs/hypomnesis/latest/x86_64-pc-windows-msvc/hypomnesis/spill/index.html . Accessed 24 October 2026. Living documentation; precise feature-introduction dates not independently verified.

[16] J. Ravi, T. Nguyen, H. Zhou and M. Becchi. PILOT a Runtime System to Manage Multi-tenant GPU Unified Memory Footprint. IEEE HiPC, 2021. DOI 10.1109/HiPC53243.2021.00063. https://hzhou.wordpress.ncsu.edu/files/2022/12/hipc21.pdf . Accessed 24 October 2026.

[17] Y. Xu, Y. Wang, N. Ren, Y. Chen and D. Zhuo. Nixie Efficient Transparent Temporal Multiplexing for Consumer GPUs. arXiv:2601.11743, 16 January 2026. https://arxiv.org/abs/2601.11743 . Preprint accessed 24 October 2026.

[18] Q. Chen et al. CONCUR High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control. arXiv:2601.22705, 30 January 2026. https://arxiv.org/abs/2601.22705 . Preprint accessed 24 October 2026.

[19] M. Lin, K. Zhou and P. Su. DrGPUM Guiding Memory Optimization for GPU-Accelerated Applications. ASPLOS Volume 3, 2023. DOI 10.1145/3582016.3582044. https://dl.acm.org/doi/10.1145/3582016.3582044 . Author abstract accessed 24 October 2026.

[20] C. Curtsinger and E. D. Berger. Coz Finding Code that Counts with Causal Profiling. SOSP, 2015, pp. 184–197. DOI 10.1145/2815400.2815409. https://arxiv.org/abs/1608.03676 . Author preprint accessed 24 October 2026.

[21] R. Krishna, M. S. Iqbal, M. A. Javidian, B. Ray and P. Jamshidi. CADET Debugging and Fixing Misconfigurations using Counterfactual Reasoning. arXiv:2010.06061v2, 2021 revision of the 2020 preprint. https://arxiv.org/abs/2010.06061 . Accessed 24 October 2026.

[22] GPUStack contributors. GGUF Parser. Official repository and memory-estimation documentation. https://github.com/gpustack/gguf-parser-go . Repository created 28 May 2024; current documentation accessed 24 October 2026. Individual feature introduction dates not audited.

[23] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023. DOI 10.1145/3600006.3613165. https://arxiv.org/abs/2309.06180 . Accessed 24 October 2026.

[24] G. Wang, R. Shen, J. Chen, Y. Xiong and L. Zhang. Probabilistic Delta Debugging. ESEC/FSE, 2021. DOI 10.1145/3468264.3468625. https://xiongyingfei.github.io/papers/FSE21a.pdf . Author manuscript accessed 24 October 2026.

[25] J. W. Buck and S. Perugini. An Interactive Graphical CPU Scheduling Simulator for Teaching Operating Systems. arXiv:1812.05160, 2018; revised 2019. https://arxiv.org/abs/1812.05160 . Accessed 24 October 2026.

[26] P. Elvinger, F. Strati, N. E. Jerger and A. Klimovic. Understanding GPU Resource Interference One Level Deeper. SoCC, 2025. DOI 10.1145/3772052.3772270. https://arxiv.org/abs/2501.16909 . Accessed 24 October 2026.

[27] xptea. VGPU-Mon. Official Windows GPU process-monitor repository, created 13 July 2026. https://github.com/xptea/VGPU-Mon . Current documentation accessed 24 October 2026; exact feature chronology not independently verified.